

Low-Level Design

for

Zero-Day Scanner for Agriculture Web Portals

Giridhar Pai

23BCCED015

BCA in Cyber Security, Ethical Hacking & Digital Forensics with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

11-04-2026

Under the guidance of

Mr. Vikyath

Lecturer, Computer Science Department

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to



YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATT, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction
 - 1.1 Scope of the Document
 - 1.2 Intended Audience
 - 1.3 System Overview
2. System Design
 - 2.1 Application Architecture Diagram
 - 2.2 Process Flow (Dispatch Lifecycle) – Sequence Diagram
 - 2.3 Information Flow (Telemetry Engine) – Data Flow Diagram
 - 2.4 Components Design (Detailed)
 - 2.5 Key Design Considerations
 - 2.6 API Catalogue
3. Data Design
 - 3.1 Entity-Relationship (ER) Diagram
 - 3.2 Data Model (Schema & Encryption)
 - 3.3 Data Access Mechanism
 - 3.4 Data Retention & Archival Policies
 - 3.5 Data Migration & Seeding
4. Interfaces
 - 4.1 User Interface (CLI & Launcher)
 - 4.2 Internal API Contracts
5. State and Session Management
6. Caching Strategy
7. Non-Functional Requirements
 - 7.1 Security Aspects
 - 7.2 Performance Aspects
 - 7.3 Availability & Recovery
8. Conclusion

1. Introduction

1.1 Scope of the Document

This Low-Level Design (LLD) defines the low-level implementation logic for Zero-Day Scanner for Agriculture Web Portals (Tejas Raksha) , an automated security vulnerability scanner for agriculture web portals. It covers component breakdown, data models, internal API contracts, state management, caching, and non-functional requirements at a level sufficient for developers to implement and maintain the system.

1.2 Intended Audience

- **System Developers** – to understand module interactions and implement/extend features.
- **Security Auditors** – to verify detection logic and test coverage.
- **DevOps Engineers** – to deploy and configure the scanner in CI/CD pipelines.
- **Project Maintainers** – to extend plugin-based architecture.

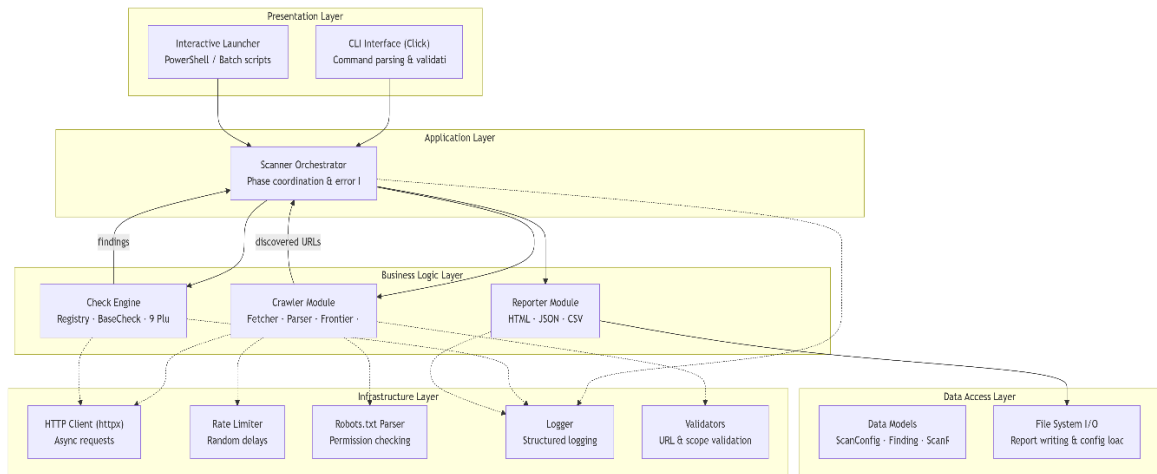
1.3 System Overview

Zero-Day Scanner for Agriculture Web Portals is a Python-based, asynchronous security scanner that enforces:

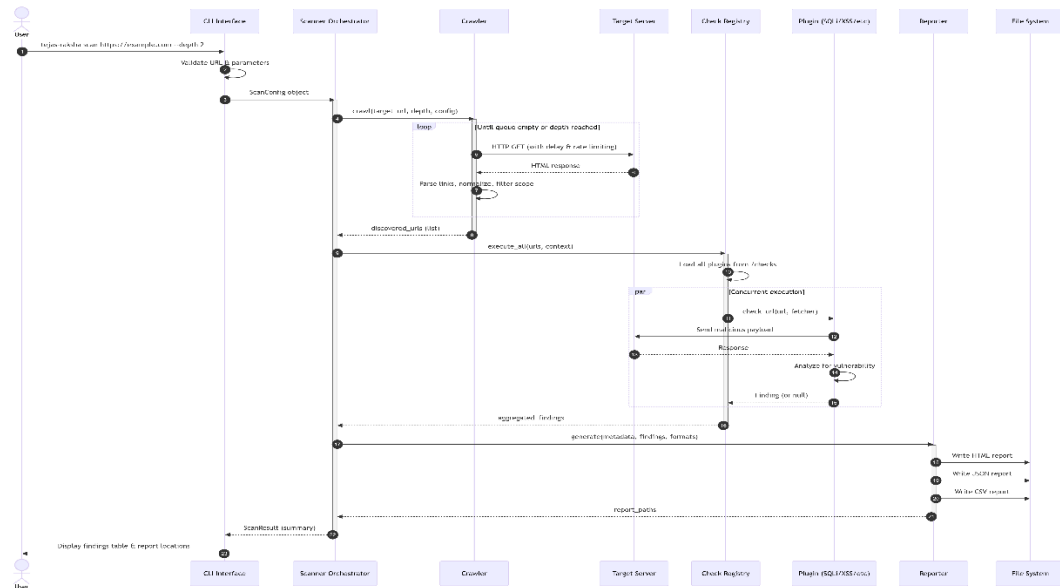
- Automated web crawling with configurable depth and scope control.
- Plugin-based detection for 9 vulnerability classes (SQLi, XSS, exposed files, directory listing, missing security headers, server info disclosure, error leakage, HTTPS misconfig, open redirect).
- Multi-format reporting (HTML, JSON, CSV).
- Rate limiting and robots.txt compliance.
- HTTP Basic/Digest authentication support.
- The system is designed for authorized security testing of agriculture portals and does not perform exploitation or automated remediation.

2. System Design

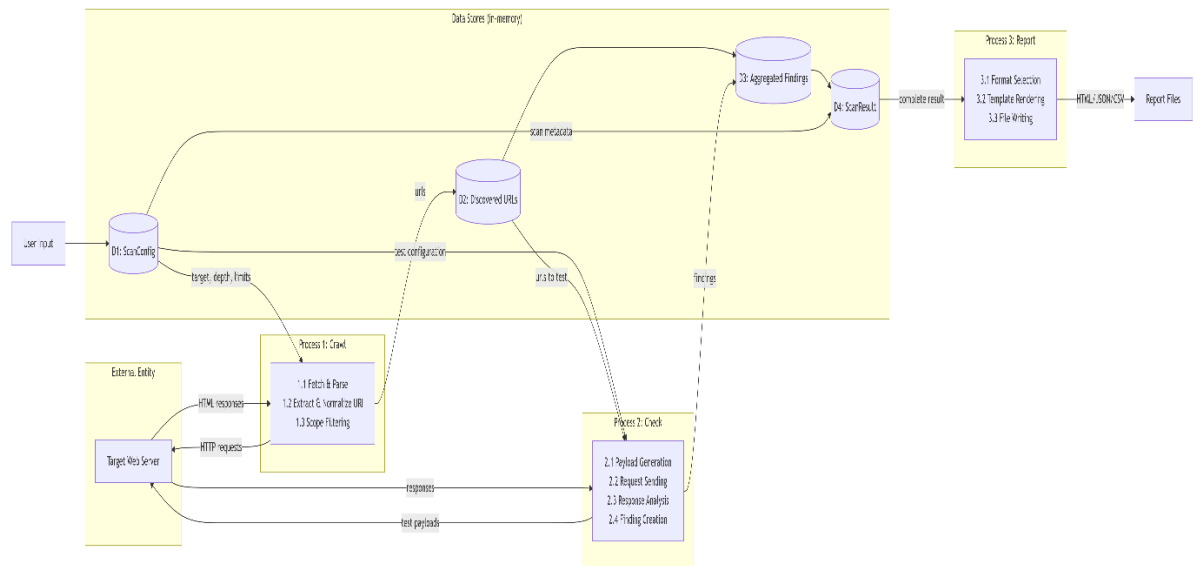
2.1 Application Design



2.2 Process Flow



2.3 Information Flow



2.4 Components Design

Component	File	Responsibilities
CLI Interface	<code>cli.py</code>	Parse arguments, validate input, invoke scanner, display summary
Scanner Orchestrator	<code>scanner.py</code>	Coordinate crawl → check → report phases, error handling, stats
Crawler Module	<code>crawler/</code>	Discover URLs with depth control, rate limiting, robots.txt
Check Engine	<code>checks/__init__.py</code>	Plugin discovery, execution, context management
SQL Injection Check	<code>checks/sqli.py</code>	Detect SQL injection via error-based payloads
XSS Check	<code>checks/xss.py</code>	Detect reflected XSS
Sensitive Files Check	<code>checks/sensitive_files.py</code>	Request common sensitive paths

Security Headers Check	<code>checks/security_headers.py</code>	Verify CSP, HSTS, X-Frame-Options, etc.
Reporter Module	<code>reporter/</code>	Generate HTML, JSON, CSV reports
Utilities	<code>utils/</code>	Rate limiting, robots parsing, validation, logging

2.5 Key Design Considerations

1. **Separation of Concerns** – Crawling, detection, and reporting are independent modules. New checks can be added without modifying core logic.
2. **Asynchronous Concurrency** – Uses `asyncio` + `httpx` for efficient concurrent requests. Semaphores limit simultaneous connections to respect target servers.
3. **Plugin Architecture** – Checks are discovered dynamically by scanning the `checks/` directory. Each check inherits from `BaseCheck` and implements `check_url()`.
4. **Respectful Scanning** – Implements:
 - Random delays between requests (configurable min/max)
 - robots.txt parser to avoid disallowed paths
 - Configurable concurrency limit (default 5)
5. **Error Isolation** – Failure in one check does not crash the entire scan. Exceptions are caught, logged, and scan continues.
6. **Defense in Depth (for scanner integrity)** –
 - No remote code execution from target responses.
 - Output files written only to user-specified directory.
 - Input validation on all user-supplied parameters.

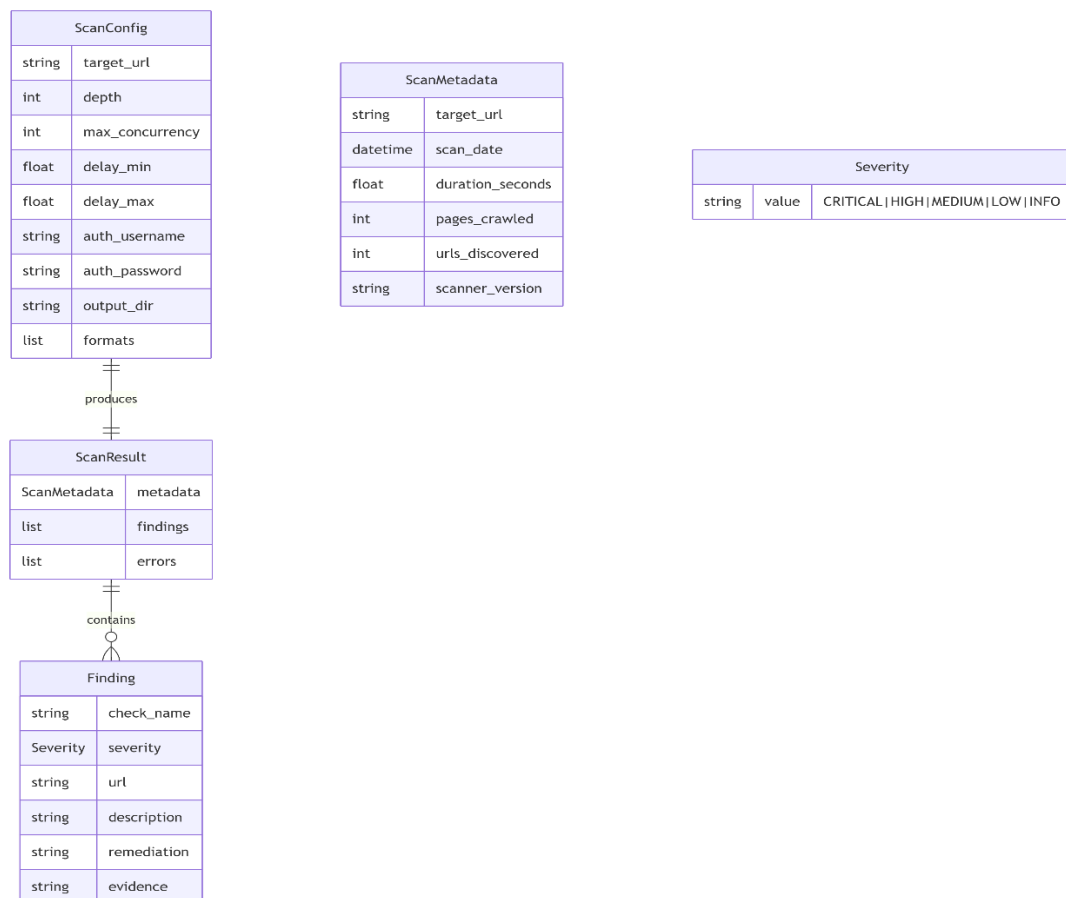
2.6 API Catalogue

"Endpoint" (Method)	Component	Input	Output
<code>Scanner.run()</code>	Orchestrator	<code>ScanConfig</code>	<code>ScanResult</code>

<code>CrawlerController.crawl(start_url)</code>	Crawler	<code>str</code>	<code>List[str]</code>
<code>CheckRegistry.execute_all(context)</code>	Check Engine	<code>CheckContext</code>	<code>List[Finding]</code>
<code>BaseCheck.check_url(url, fetcher)</code>	Individual Check	<code>str, Fetcher</code>	<code>Optional[Finding]</code>
<code>ReportGenerator.generate(result, formats)</code>	Reporter	<code>ScanResult, List[str]</code>	<code>Dict[str, Path]</code>

3. Data Design

3.1 Entity-Relationship (ER) Diagram



3.2 Data Model (Schema & Encryption)

ScanConfig

Field	Type	Description
target_url	str	Base URL to scan
depth	int	Crawl depth (1–3)
max_concurrency	int	Concurrent requests
delay_min	float	Minimum delay (seconds)
delay_max	float	Maximum delay (seconds)
auth_username	Optional[str]	HTTP Basic auth
auth_password	Optional[str]	HTTP Basic auth
output_dir	Path	Report output directory
formats	List[str]	e.g., ["html", "json", "csv"]

Finding

Field	Type	Description
check_name	str	Name of detection plugin
severity	Severity (Enum)	CRITICAL, HIGH, MEDIUM, LOW, INFO
url	str	Vulnerable URL
description	str	Human-readable description
remediation	str	Suggestion to fix vulnerability
evidence	Optional[str]	Snippet of response evidence

ScanResult

Field	Type	Description
metadata	ScanMetadata	Target, duration, pages crawled, version
findings	List[Finding]	All detected vulnerabilities
errors	List[str]	Any errors encountered

Severity Enum – CRITICAL, HIGH, MEDIUM, LOW, INFO

3.3 Data Access Mechanism

- No database used. All data is held in memory as Python objects.
- Reports are written to the filesystem via standard file I/O.
- Configuration is read from YAML (optional) or CLI arguments.
- Logs are written to console and optionally to a log file.

3.4 Data Retention & Archival Policies

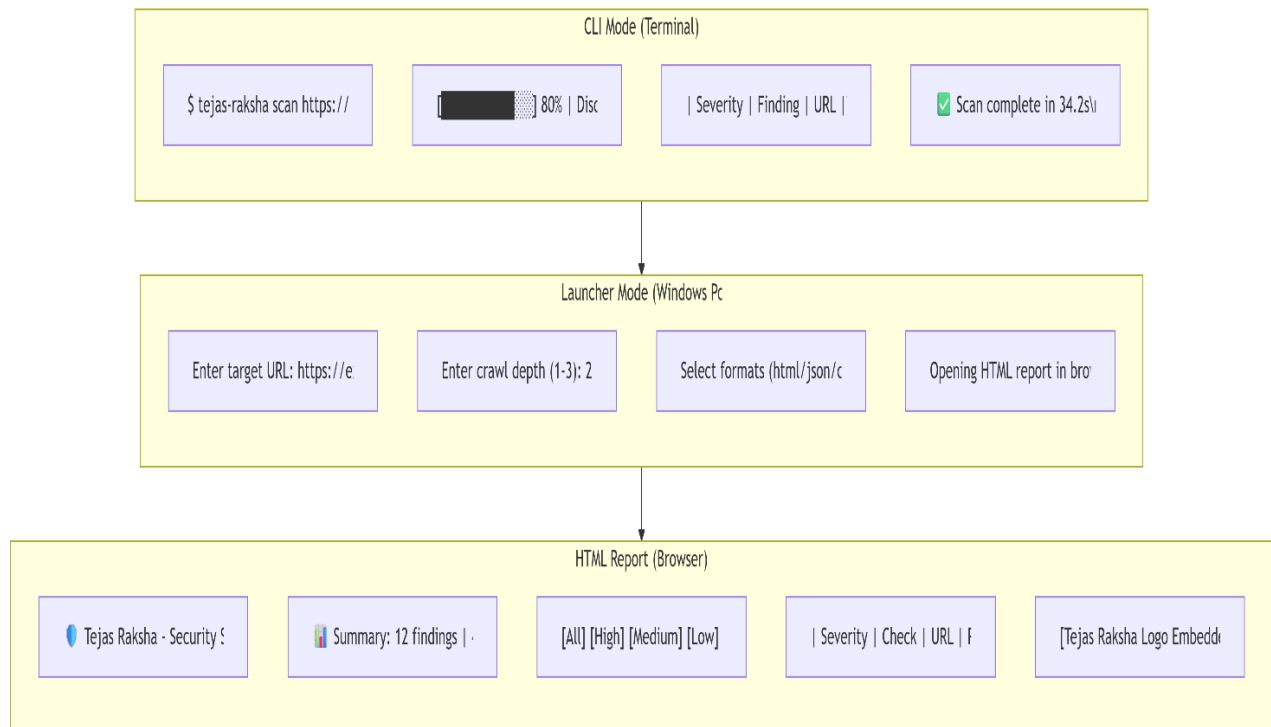
Data Entity	Retention	Mechanism
Scan reports (HTML/JSON/CSV)	User-managed	Files stored in output directory; no auto-delete
Log files	User-managed	Rotating file handler (optional)
Temporary scan data	Scan duration only	In-memory; discarded after report generation

3.5 Data Migration & Seeding

- Not applicable – no persistent database. For testing, a seed_test_targets.py script creates a list of deliberately vulnerable test URLs (e.g., testphp.vulnweb.com) for validation.

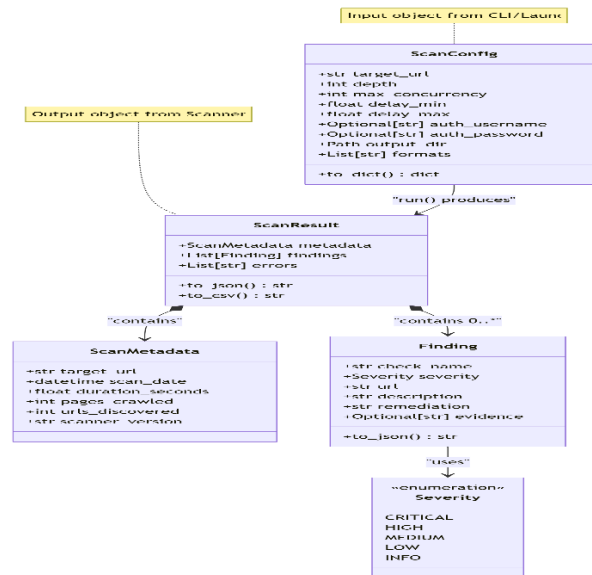
4. Interfaces

4.1 User Interface (UI) Layout

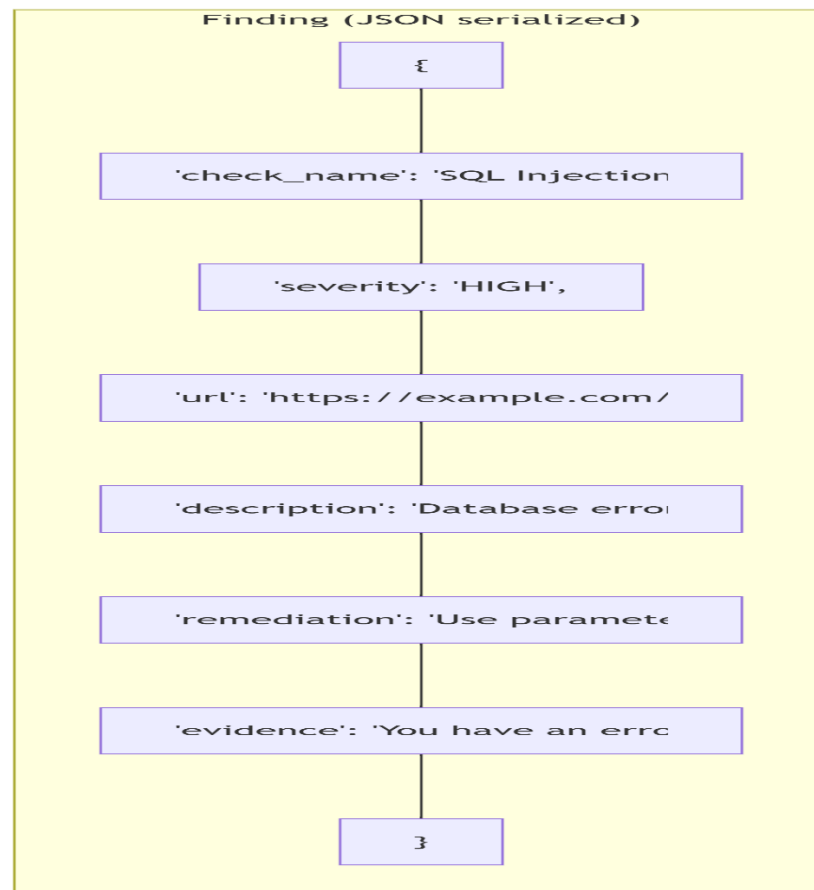


4.2 API Contracts (Request / Response Structure)

- **Internal API Data Flow (Class/Object View)**



Finding Object (JSON Serialized) – as a Diagram



5. State and Session Management

- No persistent session across scans. Each scan is independent.
- Scan state is maintained in memory within the Scanner orchestrator:
 - **discovered_urls**: Set[str]
 - **findings**: List[Finding]
 - **errors**: List[str]
 - start_time, end_time
- **Crawler state**:
 - visited: Set[str] – prevents re-crawling
 - frontier: deque – URLs to visit with depth
- **Rate limiter state** – tracks last request time per domain (not cross-domain).
- **No authentication session** – each request is independent (stateless HTTP).
-

6. Caching

Cache Type	TTL	Purpose
robots.txt cache	1 hour	Avoid fetching robots.txt before every request
DNS resolution	Per request	Relies on httpx internal cache
Response body	None	Never cached; each test uses fresh request

7. Non-Functional Requirements

7.1 Security Aspects

Control	Implementation	Control
Input validation	URL validation, depth range checks, path traversal prevention	Input validation
No code execution from target	Responses are parsed as text/HTML, never executed	No code execution from target

Safe file writing	Output written only to user-specified directory; no symlink following	Safe file writing
Logging	No secrets (passwords, tokens) written to logs	Logging
Dependency scanning	Uses <code>safety</code> or <code>pip-audit</code> in CI	Dependency scanning

7.2 Performance Aspects

Metric	Target
Pages per second	~10–20 with default concurrency (respectful)
Scan of 100 pages	Under 2 minutes
Memory usage	< 200 MB for typical scan
Concurrent connections	Configurable (default 5)

Optimizations:

- Asynchronous I/O for all network requests.
- Semaphore-based connection limiting.
- Early termination of crawling when depth limit reached.

7.3 Availability & Recovery

- **No external dependencies** – runs offline except target access.
- **Retry logic** – Failed requests retried up to 2 times with exponential backoff.
- **Graceful degradation** – If crawler fails, scanner still runs checks on any discovered URLs. If a check crashes, others continue.
- **No single point of failure** – Stateless design; can be re-run at any time.

Conclusion

The Low-Level Design (LLD) for Zero-Day Scanner for Agriculture Web Portals (Tejas Raksha) provides a detailed technical blueprint for an automated security vulnerability scanner tailored to agriculture web portals. The design covers:

- Asynchronous crawling with respectful rate limiting and robots.txt compliance.
- A plugin-based detection engine for nine OWASP Top 10 vulnerabilities.
- Multi-format reporting (HTML, JSON, CSV) with professional presentation.
- Clear separation of concerns, error isolation, and extensibility.